



UNIVERSIDAD DE CASTILLA-LA MANCHA ESCUELA SUPERIOR DE INFORMÁTICA

 Grado en Ingeniería Informática

Practica 0

Sistemas de Información Empresariales Avanzados

Curso 2025-2026

Autora:

Sara Hidalgo Salas.

Fecha: 09/02/2026

Índice

1. Mi primer programa en Python.....	5
1. Escribe un programa que muestre por pantalla “Hola mundo”	5
2. Tipos básicos.	5
2. Asigna a distintas variables valores de los siguientes tipos: cadena, entero, entero largo, real, real en notación científica. Muestra por pantalla su valor y su tipo de dato	5
3. Utiliza los valores a=3 y b=5.0 para realizar las operaciones aritméticas habituales.....	5
4. Dados los valores a=3 y c=5, realiza las operaciones de división y división entera. Fuerza a que uno de los operandos sea un número real y vuelve a hacer las divisiones.....	6
5. Crea y muestra 3 cadenas especiales: 1) con caracteres Unicode, 2) que muestre por pantalla \n, 3) una cadena definida en dos líneas.....	6
6. Crea las cadenas a="aaA" y b="bbB" y realiza operaciones de suma y multiplicación sobre ellas	7
7. Define dos variables booleanas (una verdadera y otra falsa) y muestra el resultado de realizar las operaciones and, or y not	7
8. Define las variables a=3 y b=5 y muestra el resultado de comparar ambos valores.....	7
3. Colecciones.....	8
9. Crea una lista que contenga una cadena, un entero, un booleano, una lista y un real. Muestra: la lista completa, el primer elemento de la lista y el primero del elemento lista (sublista), el último elemento (utilizando índice negativo), los elementos 0 a 2, los elementos de la lista de 2 en 2.....	8
10. Sobre la misma lista modifica el elemento booleano y el primer elemento de la sublista y muestra el resultado. Modifica ahora simultáneamente los dos primeros elementos de la lista y muestra el resultado	9
11. Crea una tupla con 3 elementos. Muestra la tupla completa y el segundo elemento. Intenta modificar el primer elemento de la tupla.....	9
12. Crea un diccionario que asocie nombres de personas con nombres de ciudades. Muéstralo. Muestra la ciudad asociada al segundo nombre. Modifica la ciudad asociada al primer nombre.	10
4. Control de flujo.....	10
13. Escribe una sentencia if que compare un valor con 10 y muestre el mensaje correspondiente.	10
14. Dadas dos variables (una par y la otra impar), asigna a otras dos variables las cadenas“par”o“impar”según el caso utilizando la asignación condicional.....	11
15. Mostrar los valores entre 20 y 1.....	11
16. Almacenar los valores entre 20 y 1 en una lista y mostrarla (ayuda: se pueden concatenar listas con el operador +)	12
17. Solicita una palabra por teclado hasta que se introduzca la palabra“fin”	12
18. Mediante un for recorre la lista creada anteriormente e imprime cada elemento	12
5. Funciones	13

19. Escribe una función que muestre la suma de dos números (o cadenas). Aplica la función a dos números y dos cadenas	13
20. Escribe una función que imprima los parámetros que se la han pasado (número variable) de dos maneras: 1) uno en cada línea y 2) como una tupla.....	14
21. Escribe una función que retorne la suma y el producto de dos números.....	14
6. Orientación a objetos	15
22. Crea una clase “contador” que admita un valor inicial y un rango de valores extremos (menor, mayor). Si el valor inicial está fuera del rango dado se reasignará el valor inicial al más cercano de los extremos. Se definirán dos métodos “incrementa” y decrementa” que realizarán las operaciones indicadas pero manteniendo el contador dentro del rango. Por defecto el incremento será de 1 salvo que se pase por parámetro un valor distinto.....	15
23. Crea una jerarquía de clases Persona ⇒ Empleado ⇒ Jefe donde Persona tiene los atributos nombre y edad (y los métodos para obtenerlos), Empleado añade el atributo sueldo, y Jefe es un empleado con un sueldo prefijado. Todos los atributos deben ser privados.	16
7. Revisitando objetos.	17
24. Define un diccionario y utiliza el método get para obtener el valor asociado a una clave. Si la clave no existe devuelve la frase “No está”. Muestra el diccionario completo, solo las claves y solo los valores.....	17
25. Dada una cadena reemplaza las ocurrencias de “a” por “A”. Separa una cadena en subcadenas cuyo carácter de separación es “a”. Convierte una cadena a mayúsculas.	18
26. Dada una lista de 3 elementos, inserta un nuevo elemento en la segunda posición y obtén su posición. Invierte la lista y vuelve a mostrar la posición. Ordena la lista.	18
8. Programación funcional	19
27. Crea una función que admita como parámetro las cadenas “+” o “*” y retorne un puntero a la función de suma o producto respectivamente. Invoca dichas funciones a través del puntero obtenido.	19
28. Utiliza map para sumar los elementos en la misma posición de dos listas.....	19
29. Utiliza filter para obtener los números mayores que 2 de una lista	20
30. Utiliza reduce para obtener el producto de todos los elementos de una lista	20
31. Utiliza “comprensión de listas” para sumar los elementos en la misma posición de dos listas.....	20
32. Utiliza “comprensión de listas” para obtener los números mayores que 2 de una lista ...	21
33. Utiliza un generador para mostrar por pantalla los múltiplos de 5 entre 0 y 50. Utiliza el mismo generador para mostrar los múltiplos de 3 entre 4 y 31.....	21
9. Excepciones	21
34. Captura el error al intentar acceder a un índice de una lista que no existe. Lo mismo para la clave de un diccionario.	21
10. Módulos y paquetes.....	22
35. Crea un módulo con el generador definido en el ejercicio 33 de tal manera que pueda invocarse tanto desde otro programa como directamente. En caso de invocarlo directamente	

deberá mostrar el mensaje “Generador de múltiplos” y mostrar por pantalla en una línea los múltiplos de 5 entre 0 y 30.....	22
36. Crea un paquete que contenga el módulo recién creado y úsalo.....	22
11. Entrada/salida y ficheros.....	23
37. Lee números del teclado hasta que se introduzca un 0. Si no se introduce un número indica el error mediante una excepción. Al introducir un 0 muestra el mensaje “Fin” capturando la excepción SystemExit.	23
38. Muestra en una sola línea los parámetros pasados como argumento (no incluir el nombre del programa).	24
39. Obtén el número de líneas que tiene un fichero y muestra la segunda línea.....	24
12. Expresiones regulares.....	25
40. Muestra las líneas de un fichero que concuerdan con una ER pasada como parámetro. Si no se pasa ningún parámetro se mostrará un mensaje de error.	25

1. Mi primer programa en Python.

1. Escribe un programa que muestre por pantalla "Hola mundo"

```
#1. Escribe un programa que muestre por pantalla "Hola mundo"
```

```
print("hola mundo")
```

```
hola mundo
```

2. Tipos básicos.

2. Asigna a distintas variables valores de los siguientes tipos: cadena, entero, entero largo, real, real en notación científica.

Muestra por pantalla su valor y su tipo de dato

```
#2. Asigna a distintas variables valores de los siguientes tipos: cadena, entero, entero largo, real, real en notación científica. Muestra por pantalla su valor y su tipo de dato
```

```
c = "Hola Mundo"
e = 5
e_real = 5.5
e_real_cientifico = 0.1e-3

print("El valor de c es: " + c) #de esta forma concatenamos la variable que es una cadena de texto
print(f"El valor de e es: {e}") #de esta forma no hace falta concatenar la variable, se puede escribir directamente
print(f"El valor de e_real es: {e_real}")
print(f"El valor de e_real_cientifico es: {e_real_cientifico}")
```

```
El valor de c es: Hola Mundo
El valor de e es: 5
El valor de e_real es: 5.5
El valor de e_real_cientifico es: 0.0001
```

3. Utiliza los valores a=3 y b=5.0 para realizar las operaciones aritméticas habituales

```
#3. Utiliza los valores a=3 y b=5.0 para realizar las operaciones aritméticas habituales
```

```
a=3
b=5.0

print(f"la suma de a y b es: {a+b}")
print(f"la resta de a y b es: {a-b}")
print(f"la multiplicacion de a y b es: {a*b}")
print(f"la division de a y b es: {a/b}")
print(f"el resto de a y b es: {a%b}")
print(f"la potencia de a y b es: {a**b}")
```

```
la suma de a y b es: 8.0
la resta de a y b es: -2.0
la multiplicacion de a y b es: 15.0
la division de a y b es: 0.6
el resto de a y b es: 3.0
la potencia de a y b es: 243.0
```

4. Dados los valores a=3 y c=5, realiza las operaciones de división y división entera. Fuerza a que uno de los operandos sea un número real y vuelve a hacer las divisiones.

```
#4. Dados los valores a=3 y c=5, realiza las operaciones de división y división entera. Fuerza a que uno
#de los operandos sea un número real y vuelve a hacer las divisiones.

a=3
c=5

print(f"la division de a y c es: {a/c}")
print(f"la division entera de a y c es: {a//c}")

#forzamos a que uno de los operandos sea un número real
print(f"la division forzando a que a sea un número real es: {float(a)/c}")
print(f"la division entera forzando a que a sea un número real es: {float(a)//c}")
```

```
la division de a y c es: 0.6
la division entera de a y c es: 0
la division forzando a que a sea un número real es: 0.6
la division entera forzando a que a sea un número real es: 0.0
```

5. Crea y muestra 3 cadenas especiales: 1) con caracteres Unicode, 2) que muestre por pantalla `\n`, 3) una cadena definida en dos líneas.

```
#5. Crea y muestra 3 cadenas especiales: 1) con caracteres unicode, 2) que muestre por pantalla \n, 3)
#una cadena definida en dos lineas.

unicode=u"äöè"
raw=r"\n"
multilinea="""esto es multilínea
y sigue siendo multilínea"""

print(f"la cadena con caracteres unicode es: {unicode}")
print(f"la cadena con la representacion del salto de linea es: {raw}")
print(f"la cadena multilínea es: {multilinea}")
```

```
la cadena con caracteres unicode es: äöè
la cadena con la representacion del salto de linea es: \n
la cadena multilínea es: esto es multilínea
y sigue siendo multilínea
```

6. Crea las cadenas a="aaA" y b="bbB" y realiza operaciones de suma y multiplicación sobre ellas

```
#6. Crea las cadenas a="aaA" y b="bbB" y realiza operaciones de suma y multiplicación sobre ellas
```

```
a="aaA"
b="bbB"

print(f"la suma de a y b es: {a+b}")
print(f"la multiplicacion de a por 3 es: {a*3}")
print(f"la multiplicacion de b por 2 es: {b*2}")
```

```
la suma de a y b es: aaAbbB
la multiplicacion de a por 3 es: aaAaaAaaA
la multiplicacion de b por 2 es: bbBbbB
```

7. Define dos variables booleanas (una verdadera y otra falsa) y muestra el resultado de realizar las operaciones and, or y not

```
#7. Define dos variables booleanas (una verdadera y otra falsa) y muestra el resultado de realizar las operaciones and, or y not
```

```
verdadero=True
falso=False

print(f"el resultado de la operacion AND entre verdadero y falso es: {verdadero and falso}")
print(f"el resultado de la operacion OR entre verdadero y falso es: {verdadero or falso}")
print(f"el resultado de la operacion NOT sobre verdadero es: {not verdadero}")
print(f"el resultado de la operacion NOT sobre falso es: {not falso}")
```

```
el resultado de la operacion AND entre verdadero y falso es: False
el resultado de la operacion OR entre verdadero y falso es: True
el resultado de la operacion NOT sobre verdadero es: False
el resultado de la operacion NOT sobre falso es: True
```

8. Define las variables a=3 y b=5 y muestra el resultado de comparar ambos valores

```
#8. Define las variables a=3 y b=5 y muestra el resultado de comparar ambos valores
```

```
a=3
b=5

print(f"¿a es igual que b? {a==b}")
print(f"¿a es distinto a b? {a!=b}")
print(f"¿a es menor que b? {a<b}")
print(f"¿a es mayor que b? {a>b}")
print(f"¿a es menor o igual que b? {a<=b}")
print(f"¿a es mayor o igual que b? {a>=b}")
```

```
¿a es igual que b? False
¿a es distinto a b? True
¿a es menor que b? True
¿a es mayor que b? False
¿a es menor o igual que b? True
¿a es mayor o igual que b? False
```

3. Colecciones.

9. Crea una lista que contenga una cadena, un entero, un booleano, una lista y un real. Muestra: la lista completa, el primer elemento de la lista y el primero del elemento lista (sublista), el último elemento (utilizando índice negativo), los elementos 0 a 2, los elementos de la lista de 2 en 2.

```
#9. Crea una lista que contenga una cadena, un entero, un booleano, una lista y un real. Muestra: la lista completa,
#el primer elemento de la lista y el primero del elemento lista (sublista), el último elemento
#(utilizando índice negativo), los elementos 0 a 2, los elementos de la lista de 2 en 2

lista = ["cadena", 3, True, [1, 2], 3.14]
print(f"la lista completa es la siguiente: {lista}")

primer_elemento = lista[0]
print(f"el primer elemento de la lista es: {primer_elemento}")
#print(f"el primer elemento de la lista es: {lista[0]}")//otra forma de hacerlo mas directo

primer_elemento_sublista = lista[3][0]
print(f"el primer elemento de la sublista es: {primer_elemento_sublista}")
#print(f"el primer elemento de la sublista es: {lista[3][0]}")//otra forma de hacerlo mas directo

ultimo_elemento = lista[-1]
print(f"el ultimo elemento utilizando indice negativo es: {ultimo_elemento}")
#print(f"el ultimo elemento utilizando indice negativo es: {lista[-1]}") //otra forma de hacerlo mas directo

elementoInicioFin = lista[0:3]#inicio:fin el fin no se incluye
print(f"los elementos de 0 a 2 son: {elementoInicioFin}")

elementoDos_Dos = lista[0:5:2] #se podria poner vacio el inicio y fin pq detecta cual es el inicio y fin automaticame
print(f"los elementos de dos en dos son los siguientes: {elementoDos_Dos}")
```

```
la lista completa es la siguiente: ['cadena', 3, True, [1, 2], 3.14]
el primer elemento de la lista es: cadena
el primer elemento de la sublista es: 1
el ultimo elemento utilizando indice negativo es: 3.14
los elementos de 0 a 2 son: ['cadena', 3, True]
los elementos de dos en dos son los siguientes: ['cadena', True, 3.14]
```

10. Sobre la misma lista modifica el elemento booleano y el primer elemento de la sublista y muestra el resultado. Modifica ahora simultáneamente los dos primeros elementos de la lista y muestra el resultado

```
#10. Sobre la misma lista modifica el elemento booleano y el primer elemento de la sublista y muestra  
#el resultado. Modifica ahora simultáneamente los dos primeros elementos de la lista y muestra el  
#resultado
```

```
lista = ["cadena", 3, True, [1, 2], 3.14]  
print(f"la lista original es: {lista}")
```

```
lista[2] = False  
print(f"ahora la nueva lista es: {lista}")
```

```
lista[3][0] = 6  
print(f"ahora la nueva lista es: {lista}")
```

```
lista[0:2] = [8, 9]  
print(f"ahora la nueva lista es: {lista}")
```

```
la lista original es: ['cadena', 3, True, [1, 2], 3.14]  
ahora la nueva lista es: ['cadena', 3, False, [1, 2], 3.14]  
ahora la nueva lista es: ['cadena', 3, False, [6, 2], 3.14]  
ahora la nueva lista es: [8, 9, False, [6, 2], 3.14]
```

11. Crea una tupla con 3 elementos. Muestra la tupla completa y el segundo elemento. Intenta modificar el primer elemento de la tupla.

```
#11. Crea una tupla con 3 elementos. Muestra la tupla completa y el segundo elemento. Intenta modificar el primer elemento de la tupla.
```

```
tupla = (1, True, "cadena")  
print(f"la tupla completa es: {tupla}")  
  
segundo_elemento = tupla[1]  
print(f"el segundo elemento de la tupla es: {segundo_elemento}")  
  
#no se pueden modificar las tuplas
```

```
la tupla completa es: (1, True, 'cadena')  
el segundo elemento de la tupla es: True
```

12. Crea un diccionario que asocie nombres de personas con nombres de ciudades. Muéstralo. Muestra la ciudad asociada al segundo nombre. Modifica la ciudad asociada al primer nombre.

```
#12. Crea un diccionario que asocie nombres de personas con nombres de ciudades. Muéstralo. Muestra la ciudad asociada al segundo nombre. Modifica la ciudad asociada al primer nombre.
```

```
diccionario = {"Juan": "Madrid", "Luis": "Barcelona"}
print(f"mi diccionario es el siguiente: {diccionario}")

print("la ciudad asociada al segundo nombre es la siguiente: " + diccionario["Luis"])

#Otra forma seria ponerlo con comilla simple
#print(f"la ciudad asociada al segundo nombre es la siguiente: {diccionario['Luis']}")

#Otra forma seria inicializando una variable
#segundo_nombre = diccionario["Luis"]
#print(f"la ciudad asociada al segundo nombre es la siguiente: {segundo_nombre}")
```

```
mi diccionario es el siguiente: {'Juan': 'Madrid', 'Luis': 'Barcelona'}
la ciudad asociada al segundo nombre es la siguiente: Barcelona
```

4. Control de flujo

13. Escribe una sentencia if que compare un valor con 10 y muestre el mensaje correspondiente.

```
#13. Escribe una sentencia if que compare un valor con 10 y muestre el mensaje correspondiente
```

```
valor = int(input("Ingresa un valor: "))

if valor == 10:
|   print(f"el valor {valor} es igual")
else:
|   print(f"el valor {valor} no es igual")

#otra forma seria
"""if valor == 10:
|   print(f"el valor {valor} es igual")
elif valor < 10:
|   print(f"el valor {valor} es menor")
else:
|   print(f"el valor {valor} es mayor")"""
```

```
Ingresa un valor: 10
el valor 10 es igual
PS C:\BAK\ASIGNATURAS UN
thon311/python.exe "c:/B
Ingresa un valor: 11
el valor 11 no es igual
```

14. Dadas dos variables (una par y la otra impar), asigna a otras dos variables las cadenas “par” o “impar” según el caso utilizando la asignación condicional.

```
#14. Dadas dos variables (una par y la otra impar), asigna a otras dos variables las cadenas "par" o "impar"
#según el caso utilizando la asignacion condicional

n1 = 2
n2 = 1

if n1 % 2 == 0 :
|   print(f"el numero {n1} es par")
else:
|   print(f"el numero {n2} es impar")

if n2 % 2 != 0:
|   print(f"el numero {n2} es impar")
else:
|   print(f"el numero {n2} es par")

#otra forma mas directo para hacer menos lineas seria con el A if C else B
"""par = "par" if (n1 % 2 == 0) else "impar"
impar = "impar" if (n2 % 2 != 0) else "par"
print(f"{n1} es {par} y {n2} es {impar}")"""
```

```
el numero 2 es par
el numero 1 es impar
```

15. Mostrar los valores entre 20 y 1

```
#15. Mostrar los valores entre 20 y 1

num = 19
while num >= 2 :
|   print(num)
|   num = num - 1

#una opcion para que se imprime toda en una linea es con end en el print
"""num = 19
while num >= 2 :
|   print(num, end=" ")
|   num = num - 1 """
```

```
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
```

16. Almacenar los valores entre 20 y 1 en una lista y mostrarla (ayuda: se pueden concatenar listas con el operador +)

```
#16. Almacenar los valores entre 20 y 1 en una lista y mostrarla (ayuda: se pueden concatenar listas con
#el operador +)

mi_lista = []
# Para ir del 20 al 1, usamos -1 para ir restando de uno en uno
for i in range(19, 0, -1):
    mi_lista.append(i) # Esto agrega el número al final de la lista
print(f"mi lista entre 20 y 1 seria: {mi_lista}") # Muestra cómo crece la lista en cada vuelta

"""#otra forma de hacerlo seria con while
num = 19
while num >= 1:
    mi_lista.append(num)
    num = num - 1
print(f"mi lista entre 20 y 1 seria: {mi_lista}")"""
```

```
mi lista entre 20 y 1 seria: [19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

17. Solicita una palabra por teclado hasta que se introduzca la palabra “fin”

```
#17. Solicita una palabra por teclado hasta que se introduzca la palabra “fin”

palabra = input("introduce una palabra: ")

while palabra != "fin":
    palabra = input("introduce otra palabra o fin para terminar: ")#guardamos la
else :
    print("se introdujo la palabra fin")
```

```
introduce una palabra: hola
introduce otra palabra o fin para terminar: adios
introduce otra palabra o fin para terminar: fin
se introdujo la palabra fin
```

18. Mediante un for recorre la lista creada anteriormente e imprime cada elemento

```
#18. Mediante un for recorre la lista creada anteriormente e imprime cada elemento

mi_lista = [19,18,17,16,15,14,13,12,11,10,9,8,7,6,5,4,3,2,1]

for i in mi_lista:# i va a ser cada numero 19,18,17... y mi_lista donde estan guardados
    print(i)

#para que sea el resultado en una sola linea podriamos utilizar dentro del print el end
""" for i in mi_lista:
    print(i, end = " dentro de estas comillas ponemos como queremos que se vea el resultado
```

19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1

5. Funciones

19. Escribe una función que muestre la suma de dos números (o cadenas). Aplica la función a dos números y dos cadenas

```
#19. Escribe una función que muestre la suma de dos números (o cadenas). Aplica la función a dos números y dos cadenas

def sum(num1, num2, cad1, cad2):
    suma = num1+num2
    sum_cad=cad1+cad2
    return suma, sum_cad

print(sum(1,2,"ho","la"))

"""#otra opcion seria imprimirlo directamente dentro del def seria asi:
def sum(num1, num2, cad1, cad2):
    suma = num1+num2
    sum_cad=cad1+cad2
    print(suma, sum_cad)
sum(1,2,"h","o")

#la otra opcion mas directa es hacerlo dentro del print
def sum(num1, num2, cad1, cad2):
    print(f"el resultado de la funcion es: {num1+num2, cad1+cad2}")
sum(1,2,"h","o")

def sum(num1, num2, cad1, cad2):
    return num1+num2, cad1+cad2
suma = sum(1,2,"h","o")
print(f"el resultado de la funcion es: {suma}")"""
```

(3, 'hola')

20. Escribe una función que imprima los parámetros que se la han pasado (número variable) de dos maneras: 1) uno en cada línea y 2) como una tupla

```
#20. Escribe una función que imprima los parámetros que se la han pasado (número variable) de dos
#maneras: 1) uno en cada línea y 2) como una tupla

def funcion(*variables):#el* es para poner todas las variables que queramos sin tener que ir definiendolas
    for i in variables:# i van a ser las variables y variables va a ser donde estan guardados cada variable
        print(i)
    print(variables)

#como hacemos el print dentro de la funcion solo es necesario llamar a la funcion y ya
funcion(1,2,3,"hola","adios", [1,2,3])
```

```
1
2
3
hola
adios
[1, 2, 3]
(1, 2, 3, 'hola', 'adios', [1, 2, 3])
```

21. Escribe una función que retorne la suma y el producto de dos números

```
#21. Escribe una función que retorne la suma y el producto de dos números

def op(num1, num2):
    suma= num1+num2
    mul = num1*num2
    return suma,mul

#cuando ponemos return en la funcion tenemos que hacer un print despues con la funcion para que se muestre por pantalla
print(f"el resultado seria : {op(4,4)}")
```

```
el resultado seria : (8, 16)
```

6. Orientación a objetos

22. Crea una clase “contador” que admita un valor inicial y un rango de valores extremos (menor, mayor). Si el valor inicial está fuera del rango dado se reasignará el valor inicial al más cercano de los extremos. Se definirán dos métodos “incrementa” y “decrementa” que realizarán las operaciones indicadas pero manteniendo el contador dentro del rango. Por defecto el incremento será de 1 salvo que se pase por parámetro un valor distinto

```
#22. Crea una clase "contador" que admita un valor inicial y un rango de valores extremos (menor,mayor).
#Si el valor inicial este fuera del rango dado se reasignar a el valor inicial al mas cercano de los extremos.
#Se definir an dos metodos "incrementa" y "decrementa" que realizar an las operaciones indicadas pero
#manteniendo el contador dentro del rango. Por defecto el incremento ser a de 1 salvo que se pase por
#par a metro un valor distinto

#Class
class contador:
    def __init__(self, valor_inicial, menor, mayor):#el valor inicial es el valor del contador
        #Atributos
        self.valor_inicial = valor_inicial
        self.menor = menor
        self.mayor = mayor

        print(f"el valor inicial es: {self.valor_inicial}")
        if self.valor_inicial < self.menor or self.valor_inicial > self.mayor: #si se cumple uno solo es OR, si se cumplen los dos AND
            print(f"El valor inicial esta fuera del rango") #para que sea fuera del rango uno debe ser menor que 0 y el otro mayor a 10 como no se pueden cumplir los dos a la vez es un or
            if self.valor_inicial < self.menor:
                self.valor_inicial = self.menor #guardamos el nuevo valor
                print(f"El nuevo valor mas cercano del valor inicial es: {self.valor_inicial}")

            elif self.valor_inicial > self.mayor:
                self.valor_inicial = self.mayor #guardamos el nuevo valor
                print(f"El nuevo valor mas cercano del valor inicial es: {self.valor_inicial}")

        #Esta opcion de ahora va a ser utilizando la media
        """media = (self.menor + self.mayor)/2
        if self.valor_inicial < media:
            self.valor_inicial = self.menor
            print(f"ahora el valor inicial es: {self.valor_inicial}")

        elif self.valor_inicial > media:
            self.valor_inicial = self.mayor
            print(f"ahora el valor inicial es: {self.valor_inicial}")"""

        elif self.valor_inicial >= self.menor and self.valor_inicial <= self.mayor: #esta dentro del rango y es and porque se deben cumplir los dos para que este dentro del rango:
            print(f"El valor inicial esta dentro del rango") #para que este dentro del rango debe ser entre 0 y 10 por lo que se debe cumplir que uno sea mayor que 0 y el otro menor que 10

    def Incrementa(self, num_sum=1): #num_sum=1 para pasarle por defecto 1
        nuevo_valor = self.valor_inicial + num_sum
        print(f"El valor del incremento es: {nuevo_valor}")
        #comprobamos si el nuevo valor esta fuera del rango
        if nuevo_valor < self.menor:
            nuevo_valor = self.menor
            print(f"Como el nuevo valor al incrementarlo esta por debajo del rango su nuevo valor mas cercano es: {nuevo_valor}")
        elif nuevo_valor > self.mayor:
            nuevo_valor = self.mayor
            print(f"Como el nuevo valor al incrementarlo esta por encima del rango su nuevo valor mas cercano es: {nuevo_valor}")
        self.valor_inicial = nuevo_valor #guardamos su valor actual despues de incrementarlo

    def Decrementa(self, num_rest=1):
        nuevo_valor = self.valor_inicial - num_rest #pensar que el valor inicial es el ultimo que se ha quedado al hacer el incremento
        print(f"El valor del decremento es: {nuevo_valor}")
        #comprobamos si el nuevo valor esta fuera del rango
        if nuevo_valor < self.menor:
            nuevo_valor = self.menor
            print(f"Como el nuevo valor al decrementarlo esta por debajo del rango su nuevo valor mas cercano es: {nuevo_valor}")
        elif nuevo_valor > self.mayor:
            nuevo_valor = self.mayor
            print(f"Como el nuevo valor al decrementarlo esta por encima del rango su nuevo valor mas cercano es: {nuevo_valor}")
        self.valor_inicial = nuevo_valor #guardamos su valor actual despues de decrementarlo

#mi_contador = contador(10, 0, 10) #Objeto
#print(mi_contador.valor_inicial)
#mi_contador.Incrementa()
#mi_contador.Decrementa()
```

```
el valor inicial es: 10
El valor inicial esta dentro del rango
El valor del incremento es: 11
Como el nuevo valor al incrementarlo esta por encima del rango su nuevo valor mas cercano es: 10
El valor del decremento es: 9
```

23. Crea una jerarquía de clases Persona ⇒ Empleado ⇒ Jefe donde Persona tiene los atributos nombre y edad (y los métodos para obtenerlos), Empleado añade el atributo sueldo, y Jefe es un empleado con un sueldo prefijado. Todos los atributos deben ser privados.

```
#23. Crea una jerarquía de clases Persona ⇒ Empleado ⇒ Jefe donde Persona tiene los atributos nombre
#y edad (y los métodos para obtenerlos), Empleado añade el atributo sueldo, y Jefe es un empleado
#con un sueldo prefijado. Todos los atributos deben ser privados.

class Persona:
    def __init__(self, nombre, edad):
        self.__nombre = nombre# __ significa que es un atributo privado
        self.__edad = edad

    def mostrar_nombre(self):
        print(f"Mi nombre es: {self.__nombre}")

    def mostrar_edad(self):
        print(f"Mi edad es de {self.__edad} años")

class Empleado(Persona):
    def __init__(self, nombre, edad, sueldo):#Inicializamos con lo que ya tenia el padre mas el atributo nuevo, ponemos sueldo pq se lo pasaremos por parametro
        super().__init__(nombre, edad)#Inicializamos con lo que ya tenia el padre es decir clase persona
        self.__sueldo = sueldo#Añadimos el nuevo atributo
    def mostrar_sueldo(self):
        print(f"El sueldo es: {self.__sueldo}€")

class Jefe(Empleado):
    def __init__(self, nombre, edad):#inicializamos con lo que tenia el padre, no ponemos el parametro de sueldo pq sera un sueldo ya fijado
        sueldo = 3000
        super().__init__(nombre, edad, sueldo)#se inicializa con lo que tiene la clase empleado

print("---- PERSONA ----")
mi_persona = Persona("Luis", 18)
mi_persona.mostrar_nombre()
mi_persona.mostrar_edad()
print()

print("---- EMPLEADO ----")
mi_empleado = Empleado("lolo", 20, 1000)
mi_empleado.mostrar_nombre()
mi_empleado.mostrar_edad()
mi_empleado.mostrar_sueldo()
print()

print("---- JEFE ----")
mi_jefe = Jefe("Jose", 40)
mi_jefe.mostrar_nombre()
mi_jefe.mostrar_edad()
mi_jefe.mostrar_sueldo()
```

```
---- PERSONA ----
Mi nombre es: Luis
Mi edad es de 18 años

---- EMPLEADO ----
Mi nombre es: lolo
Mi edad es de 20 años
El sueldo es: 1000€

---- JEFE ----
Mi nombre es: Jose
Mi edad es de 40 años
El sueldo es: 3000€
```

7. Revisitando objetos.

24. Define un diccionario y utiliza el método get para obtener el valor asociado a una clave. Si la clave no existe devuelve la frase “No está” . Muestra el diccionario completo, solo las claves y solo los valores.

```
#24. Define un diccionario y utiliza el método get para obtener el valor asociado a una clave. Si la clave no
#existe devuelve la frase "No está". Muestra el diccionario completo, solo las claves y solo los valores.

| | | | #clave #valor
diccionario = { "Nombre" : "Luis", "Apellido": "Perez", "Num" : 1}

#MOSTRAMOS CON EL METODO GET, EL VALOR ASOCIADO A LA CLAVE NOMBRE
print(diccionario.get("Nombre"))
print()

#MOSTRAMOS CON EL METODO GET QUE LA CLAVE HOLA NO ESTA EN EL DICCIONARIO
print(diccionario.get("hola", "La clave hola no esta"))
print()

#MOSTRAMOS EL DICCIONARIO COMPLETO
print(f"Mi diccionario completo es: {diccionario}")
print()

#MOSTRAMOS CON EL METODO KEYS SOLO LAS CLAVES
print(f"Mis claves son: {diccionario.keys()}")

#SI NO QUEREMOS QUE APAREZCA EL DICT_KEYS AL IMPRIMIR LO PONEMOS COMO UNA LISTA Y QUEDA:
print(f"Mis claves como lista son: {list(diccionario.keys())}")
print()

#MOSTRAMOS CON EL METODO VALUE TODOS LOS VALORES
print(f"Mis valores son: {diccionario.values()}")

#SI NO QUEREMOS QUE APAREZCA EL DICT_KEYS AL IMPRIMIR LO PONEMOS COMO UNA LISTA Y QUEDA:
print(f"Mis valores como lista son: {list(diccionario.values())}")
```

Luis

La clave hola no esta

Mi diccionario completo es: {'Nombre': 'Luis', 'Apellido': 'Perez', 'Num': 1}

Mis claves son: dict_keys(['Nombre', 'Apellido', 'Num'])

Mis claves como lista son: ['Nombre', 'Apellido', 'Num']

Mis valores son: dict_values(['Luis', 'Perez', 1])

Mis valores como lista son: ['Luis', 'Perez', 1]

DE GRADO ACREDITACIÓN UNIVERSIDAD 2025 2026 Escuela Superior de Ingeniería de Telecomunicaciones

25. Dada una cadena reemplaza las ocurrencias de “a” por “A”. Separa una cadena en subcadenas cuyo carácter de separación es “a”. Convierte una cadena a mayúsculas.

```
#25. Dada una cadena reemplaza las ocurrencias de "a" por "A". Separa una cadena en subcadenas cuyo  
#carácter de separación es "a". Convierte una cadena a mayúsculas.
```

```
#Definimos una cadena  
cadena = "una ola"  
#CON EL METODO REPLACE REEMPLAZAMOS LA a minuscula por la A MAYUSCULA  
print(cadena.replace("a","A"))  
#CON EL METODO SPLIT SEPARAMOS LA CADENA EN SUBCADENAS POR EL CARACTER QUE PASEMOS  
print(cadena.split("a"))  
#CON EL METODO UPPER PONEMOS LA CADENA EN MAYUSCULAS  
print(cadena.upper())
```

```
unA olA  
['un', ' ol', '']  
UNA OLA
```

26. Dada una lista de 3 elementos, inserta un nuevo elemento en la segunda posición y obtén su posición. Invierte la lista y vuelve a mostrar la posición. Ordena la lista.

```
#26. Dada una lista de 3 elementos, inserta un nuevo elemento en la segunda posición y obtén su posición.  
#Invierte la lista y vuelve a mostrar la posición. Ordena la lista.
```

```
#Definimos una lista  
lista = [1,2,3]  
#INSERTAMOS UN NUEVO ELEMENTO EN LA SEGUNDA POSICION  
lista.insert(2,5)  
print(lista)  
#OBTENEMOS LA POSICION DEL ELEMENTO AÑADIDO  
print(lista.index(5))  
#INVERTIMOS LA LISTA  
lista.reverse()  
print(lista)  
#OBTENEMOS LA POSICION DEL ELEMENTO  
print(lista.index(5))  
#ORDENAMOS LA LISTA  
lista.sort()  
print(lista)
```

```
[1, 2, 5, 3]  
2  
[3, 5, 2, 1]  
1  
[1, 2, 3, 5]
```

8. Programación funcional

27. Crea una función que admita como parámetro las cadenas “+” o “*” y retorne un puntero a la función de suma o producto respectivamente. Invoca dichas funciones a través del puntero obtenido.

```
#27. Crea una función que admita como parámetro las cadenas “+” o “*” y retorne un puntero a la función
#de suma o producto respectivamente. Invoca dichas funciones a través del puntero obtenido.

def cadenas(parametro):
    def suma():
        print("Esta es la funcion de suma")

    def producto():
        print("Esta es la funcion de producto")

    lang_funciones = {"+": suma, "*": producto}
    return lang_funciones[parametro]

funcion_suma = cadenas("+")
funcion_suma()

funcion_producto = cadenas("*")
funcion_producto()
```

```
Esta es la funcion de suma
Esta es la funcion de producto
```

28. Utiliza map para sumar los elementos en la misma posición de dos listas

```
#28. Utiliza map para sumar los elementos en la misma posición de dos listas

def suma(n, m):
    return n + m

l1 = [1, 2, 3]
l2 = [3, 4, 5]
l3 = map(suma, l1, l2)
print(list(l3))

"""OTRA FORMA DE HACERLO
lista1 = [1,2,3]
lista2 = [3,4,5]

#lambda es para hacer funciones simples en una linea
lista_suma = map(lambda x, y: x + y, lista1, lista2)
print(list(lista_suma))"""
```

```
l1=[1,2,3]
l2=[3,4,5]
l3=[4,6,8]
```

29. Utiliza filter para obtener los números mayores que 2 de una lista

```
#29. Utiliza filter para obtener los numeros mayores que 2 de una lista

def mayorDos(n):
    return (n>2)
l = [1,2,3,4,5,-2]
#funcion,secuencia -> nuestra l (los elementos que tiene dentro l) pasaria a ser la n
l2 = filter(mayorDos,l)
print(list(l2))
```

```
[3, 4, 5]
```

30. Utiliza reduce para obtener el producto de todos los elementos de una lista

```
#30. Utiliza reduce para obtener el producto de todos los elementos de una lista

#A partir de Python 3 reduce no está integrada en el espacio de nombres global y debe importarse desde el módulo functools.
from functools import reduce
def mul(x, y):
    return x * y
l = [1, 2, 3, 4]
l2 = reduce(mul, l)
print(l2)

"""Otra manera de importar:
import functools
def mul(x, y):
    return x * y
l = [1, 2, 3, 4]
l2 = functools.reduce(mul, l)
print(l2)"""
```

24

31. Utiliza “comprensión de listas” para sumar los elementos en la misma posición de dos listas

```
#31. Utiliza “comprensión de listas” para sumar los elementos en la misma posición de dos listas

l1 = [1, 2, 3]
l2 = [4, 5, 6]

lista_suma = [x + y for x in l1 for y in l2 if l1.index(x) == l2.index(y)]
print(list(lista_suma))

"""OTRA FORMA DE HACERLO CON LA FUNCION ZIP
l1 = [1, 2, 3]
l2 = [4, 5, 6]

#Utilizamos zip para recorrer dos listas
l3 = [x + y for x, y in zip (l1, l2)]
print(l3)"""
```

```
[5, 7, 9]
```

32. Utiliza “comprensión de listas” para obtener los números mayores que 2 de una lista

```
#32. Utiliza “comprensión de listas” para obtener los numeros mayores que 2 de una lista
```

```
lista1 = [1, 2, 3, 4, -2]
#esto quiere decir para cada n que hay en lista 1 se haga n>2
lista2 = [n for n in lista1 if n > 2]
print(lista2)
```

```
[3, 4]
```

33. Utiliza un generador para mostrar por pantalla los múltiplos de 5 entre 0 y 50. Utiliza el mismo generador para mostrar los múltiplos de 3 entre 4 y 31

```
#33. Utiliza un generador para mostrar por pantalla los m'ultiplos de 5 entre 0 y 50. Utiliza el mismo
#generador para mostrar los m'ultiplos de 3 entre 4 y 31
```

```
#Ahora lo haremos con funciones para el ejercicio 35, Un generador se usa cuando una sola función debe producir MUCHOS valores uno tras otro.
def generador_multiplos(multiplo, inicio_rango, final_rango):
    for n in range(inicio_rango, final_rango):
        if n % multiplo == 0 :
            yield n

print(list(generador_multiplos(5, 0, 51)))
print(list(generador_multiplos(3, 4, 31)))
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
[6, 9, 12, 15, 18, 21, 24, 27, 30]
```

9. Excepciones

34. Captura el error al intentar acceder a un índice de una lista que no existe. Lo mismo para la clave de un diccionario.

```
#34. Captura el error al intentar acceder a un índice de una lista que no existe. Lo mismo para la clave de
#un diccionario
```

```
lista1 = [1, 2, 3]
try:
    print(lista1[3])
except(IndexError):#Aquí ponemos que es IndexError pq es el error que nos marca pero podrian ser otros errores
    print("No existe ese indice en la lista")

diccionario = {"Nombre": "Luis"}
try:
    nombre = diccionario["Apellido"]
    print(nombre)
except(KeyError):
    print("\nNo existe esa clave en el diccionario")
```

```
No existe ese indice en la lista
```

```
No existe esa clave en el diccionario
```

10. Módulos y paquetes

35. Crea un módulo con el generador definido en el ejercicio 33 de tal manera que pueda invocarse tanto desde otro programa como directamente. En caso de invocarlo directamente deberá mostrar el mensaje “Generador de múltiplos” y mostrar por pantalla en una línea los múltiplos de 5 entre 0 y 30.

```
Ejercicio33_35_SIEA.py > ...
1 #Ejercicio 33_35
2
3 def generador_multiplos(multiplo, inicio_rango, final_rango):
4     for n in range(inicio_rango, final_rango):
5         if n % multiplo == 0:
6             yield n
7
8 #Usamos if __name__ == "__main__" para que el código dentro de este bloque solo se ejecute si el archivo se ejecuta directamente
9 if __name__ == "__main__":
10     # Código dentro de este bloque solo se ejecuta
11     # cuando ejecutamos el archivo directamente,
12     # no cuando lo importamos desde otro programa.
13     print("generador de multiplos")
14     print(list(generador_multiplos(5, 0, 31)))
```

```
/Users/pc/AppData/Local/Programs/Python/Python311/python.exe "c:/BAK/ASIGNATURAS U
mpresariales Avanzados/Laboratorio/Parte 0/Ejercicios/Ejercicio33_35_SIEA.py"
generador de multiplos
[0, 5, 10, 15, 20, 25, 30]
```

```
#35. Crea un módulo con el generador definido en el ejercicio 33 de tal manera que pueda invocarse tanto
#desde otro programa como directamente. En caso de invocarlo directamente deberá mostrar el mensaje
#"Generador de múltiplos" y mostrar por pantalla en una línea los múltiplos de 5 entre 0 y 30.

import Ejercicio33_35_SIEA

#Con el import llamamos al programa que queramos en este caso el ejercicio 33_35, lo que hace es meterse y leer e imprimir
"""Ejercicio33_35_SIEA.generador_multiplos(5, 0, 31)"""#Aquí nos metemos en el .py y lee todo el programa como no hay un p

#Si no tenemos ningún print que muestre la salida en el programa que hemos hecho el import lo tenemos que hacer aquí
print(list(Ejercicio33_35_SIEA.generador_multiplos(5,0,31)))
```

```
[0, 5, 10, 15, 20, 25, 30]
```

36. Crea un paquete que contenga el módulo recién creado y úsalo.

```
▼ paquete
> __pycache__
  ⊕ __init__.py
  ⊕ Ejercicio33_36_SIEA.py
```

```
paquete > Ejercicio33_36_SIEA.py > ...
1 #Ejercicio 33_36
2
3 def generador_multiplos(multiplo, inicio_rango, final_rango):
4     for n in range(inicio_rango, final_rango):
5         if n % multiplo == 0:
6             yield n
7
```

#36. Crea un paquete que contenga el módulo recién creado y úsalo.

```
import paquete.Ejercicio33_36_SIEA
print(list(paquete.Ejercicio33_36_SIEA.generador_multiplos(5,0,31)))
```

#Otra forma con from e import

```
"""from paquete.Ejercicio33_35_SIEA import generador_multiplos
print(list(generador_multiplos(5,0,31)))"""
```

```
PS C:\BAK\ASIGNATURAS UNIVERSIDAD 2025-2026\Segundo Cuatri\Sistemas De Informacion Empresariales Avanzados\Laboratorio\Parte 0\Ejercicios> & C:/Users/pc/AppData/Local/Programs/Python/Py
thon311/python.exe "C:/BAK/ASIGNATURAS UNIVERSIDAD 2025-2026/Segundo Cuatri/Sistemas De Informacion Empresariales Avanzados/Laboratorio/Parte 0/Ejercicios/Ejercicio36_SIEA.py"
[0, 5, 10, 15, 20, 25, 30]
```

11. Entrada/salida y ficheros

37. Lee números del teclado hasta que se introduzca un 0. Si no se introduce un número indica el error mediante una excepción. Al introducir un 0 muestra el mensaje “Fin” capturando la excepción SystemExit.

#37. Lee números del teclado hasta que se introduzca un 0. Si no se introduce un número indica el error mediante una excepción. Al introducir un 0 muestra el mensaje “Fin” capturando la excepción SystemExit.

```
try:
    num = int(input("Introduce un numero\n"))
    while num != 0:
        | num = int(input("Introduce otro numero o 0 para finalizar\n"))
        | else:
        |     exit()# Esto lanza SystemExit
except ValueError:
    | print("Error, no has introducido un numero.")
except SystemExit:
    | print("Fin")
```

#Otra version, en esta version si se escribe una cadena no se finaliza, te pide otro numero

```
"""num = int(input("Introduce un numero\n"))
T = True
while T:
    try:
        | if (num != 0):
        |     num = int(input("Introduce otro numero o 0 para finalizar\n"))
        | else:
        |     T = False
        |     exit()# Esto lanza SystemExit
    except ValueError:
        | print("Error, no has introducido un numero, vuelve a intentarlo.\n")
    except SystemExit:
        | print("Fin")
```

#Otra version con break

```
while True:
    try:
        | num = int(input("Introduce un numero o 0 para salir:\n"))
        | if num == 0:
        |     | exit()
    except ValueError:
        | print("Error, no has introducido un numero.")
    except SystemExit:
        | print("Fin")
        | break"""
```

```
Introduce un numero
1
Introduce otro numero o 0 para finalizar
2
Introduce otro numero o 0 para finalizar
3
Introduce otro numero o 0 para finalizar
4
Introduce otro numero o 0 para finalizar
0
Fin
```

38. Muestra en una sola línea los parámetros pasados como argumento (no incluir el nombre del programa).

```
#38. Muestra en una sola linea los parámetros pasados como argumento (no incluir el nombre del programa)

import sys

#esto dice si el tamaño de los argumentos es mayor que uno, pq uno va a ser el nombre del programa ya que
if len(sys.argv) > 1:
|   print("Argumentos:", sys.argv[1:])
else:
|   print("Escribe mas argumentos")

PS c:\BAK\ASIGNATURAS UNIVERSIDAD 2025-2026\Segundo Cuatri\Sistemas De Informacion Empresariales Avanzados\Laboratorio\Parte 0\Ejercicios> python .\Ejercicio38_SIEA.py
Escribe mas argumentos
PS c:\BAK\ASIGNATURAS UNIVERSIDAD 2025-2026\Segundo Cuatri\Sistemas De Informacion Empresariales Avanzados\Laboratorio\Parte 0\Ejercicios> python .\Ejercicio38_SIEA.py hola mundo que tal
Argumentos: ['hola', 'mundo', 'que', 'tal']
```

39. Obtén el número de líneas que tiene un fichero y muestra la segunda línea.

```
≡ archivo.txt
1   hola ER
2   hola mundo que tal estas
3   adios ER
```

```
#39. Obten el número de líneas que tiene un fichero y muestra la segunda línea

f = open("archivo.txt")#con open abrimos el archivo
leer = f.readlines()# readlines lee todas las lineas del archivo txt
print(f"tenemos {len(leer)} lineas")# con len imprimimos el numero de lineas que tiene el archivo txt
print(leer[1])# se imprime la posicion de la linea si tenemos 3 lineas como aqui, la posicion 1 es la linea 2
f.close()
```

```
tenemos 3 lineas
hola mundo que tal estas
```

12. Expresiones regulares

40. Muestra las líneas de un fichero que concuerdan con una ER pasada como parámetro. Si no se pasa ningún parámetro se mostrará un mensaje de error.

```
≡ archivo.txt
1  hola ER
2  hola mundo que tal estas
3  adios ER
```

```
#40. Muestra las líneas de un fichero que concuerdan con una ER pasada como parámetro. Si no se pasa
ningún parámetro se mostrará un mensaje de error.

import sys
import re

#aquí decimos: si la longitud de los argumentos es menor a 2 o sea si no se le pasa argumentos o solo el nombre del programa da error, si es 2 o mayor qu
if len(sys.argv) < 2:
    print("Error: no se ha pasado ninguna expresión regular.")
else:
    patron = sys.argv[1]# guardamos el primer argumento que sera lo que estamos buscando en el txt
    archivo = open("archivo.txt")# abrimos el archivo
    for linea in archivo:#recorremos el archivo
        if re.search(patron, linea):#buscamos en todo el archivo donde aparezca el parametro que hayamos pasado en este caso ER
            print(linea, end="")#imprimimos la linea donde aparece y con end evitamos el doble salto de linea

PS C:\BAK\ASIGNATURAS UNIVERSIDAD 2025-2026\Segundo Cuatri\Sistemas De Informacion Empresariales Avanzados\Laboratorio\Parte 0\Ejercicios> & c:/Users/pc/AppData/Local/Programs/Python/Py
thon311/python.exe "c:/BAK/ASIGNATURAS UNIVERSIDAD 2025-2026/Segundo Cuatri/Sistemas De Informacion Empresariales Avanzados/Laboratorio/Parte 0/Ejercicios/Ejercicio40_SIEA.py"
Error: no se ha pasado ninguna expresión regular.
PS C:\BAK\ASIGNATURAS UNIVERSIDAD 2025-2026\Segundo Cuatri\Sistemas De Informacion Empresariales Avanzados\Laboratorio\Parte 0\Ejercicios> python .\Ejercicio40_SIEA.py ER
hola ER
adios ER
```